

**ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ ІВАНА ФРАНКА**

**ЗВІТ
ДО ЛАБОРАТОРНОЇ РОБОТИ №5
«Обробка помилок, файли та серіалізація»
з курсу «Професійний Python»
Варіант №1 — «Імпорт та експорт студентів»**

Виконав: Лис Артур

Група: ФЕП-31

Перевірів(ла): _____

Дата: 08.09.2026

Львів — 2026

1. Мета роботи

Навчитися писати Python-код, який не падає з голим traceback і не залишає після себе пошкоджених файлів: власна ієрархія винятків із збереженою причиною (`raise ... from`), принципи fail-fast і EAFP, context managers — стандартні й власні, logging замість print, pathlib замість склеювання рядків, потокове читання CSV/JSON Lines, серіалізація у JSON/YAML/CSV, зовнішня конфігурація з валідацією, атомарний запис результату. Усе це — у вигляді модуля імпорту/експорту студентів для проєкту student-manager, з експериментальною перевіркою strict/tolerant режимів та eager/streaming читання.

2. Індивідуальний варіант і постановка задачі

Варіант №1 — «Імпорт та експорт студентів». Поля запису: student_id, name, group, grade, email. Потрібно: імпорт із CSV; перевірка ідентифікатора, імені, оцінки в межах 0–100 і базова перевірка e-mail; пропуск невалідних записів у tolerant mode; експорт у JSON; конфігурація в YAML; статистика valid/invalid.

Загальні вимоги: Path, потоковий вхід, власна exception hierarchy з chaining, with і власний context manager, logging, strict/tolerant, статистика, atomic output, обробка помилок вводу-виводу, коректне завершення при критичних помилках. Із завдань підвищеної складності реалізовано: єдиний Importer/Exporter Protocol для CSV, JSON, JSON Lines і YAML з автоматичним вибором за розширенням; окремий error CSV; RotatingFileHandler; вкладає конфігурація з версією схеми та override зі змінних середовища; атомарний експорт із резервною копією; потоковий JSON Lines; progress-статистика; transaction-like context manager; summary report; gzip; власний JSON encoder.

3. Вхідні дані

Поле	Тип	Правило
student_id	int	ціле, додатне
name	str	непорожнє, «Прізвище Ім'я» — щонайменше два слова
group	str	непорожнє; шаблон групи перевіряє модель Student із ЛР1
grade	float	число (кома допускається), $0 \leq \text{grade} \leq 100$
email	str	базова перевірка: один @, щось до нього, крапка після, без пропусків

Демонстраційний `students.csv` — дев'ять реальних студентів із набору ЛР2 (оцінка = середнє чотирьох дисциплін) і шість навмисно зіпсованих рядків: порожнє ім'я, оцінка 105, `abc` замість `id`, `wrong` замість оцінки, e-mail без `@`, група без префікса.

```
student_id,name,group,grade,email
1,Гнатишин Марта,ФЕП-31с,93.25,marta.hnatyshyn@lnu.edu.ua
2,Дзюба Остап,ФЕП-31с,79,ostap.dziuba@lnu.edu.ua
3,Вовк Тарас,ФЕП-31с,85,taras.vovk@lnu.edu.ua
4,Кравець Соломія,ФЕП-32,88,solomiia.kravets@lnu.edu.ua
5,Пасічник Ірина,ФЕП-32,96.5,iryna.pasichnyk@lnu.edu.ua
6,Мельник Андрій,ФЕП-32,73.5,andrii.melnyk@lnu.edu.ua
7,Скиба Юрій,ФЕП-33,71.25,yurii.skyba@lnu.edu.ua
8,Ковальчук Назар,ФЕП-33,82.5,nazar.kovalchuk@lnu.edu.ua
9,Тимків Богдан,ФЕП-33,90.5,bohdan.tymkiv@lnu.edu.ua
10,,ФЕП-31с,76.0,noname@lnu.edu.ua
11,Новак Петро,ФЕП-33,105,petro.novak@lnu.edu.ua
abc,Бойко Марія,ФЕП-32,85,mariia.boiko@lnu.edu.ua
13,Литвин Андрій,ФЕП-31с,wrong,andrii.lytvyn@lnu.edu.ua
14,Шевчук Олена,ФЕП-32,88.5,olena.shevchuk-at-lnu
15,Романюк Ігор,31с,79,ihor.romaniuk@lnu.edu.ua
```

4. Структура проєкту

Підпакет `storage` додано до проєкту (версія 0.5.0); файли демонстрації, результати й журнали живуть у `data/` і не потрапляють у `Git`.

```
src/
  student_manager/
    __init__.py
    cli.py
    config.py
    exceptions.py
    main.py
    models.py
    registry.py
    analytics/
      __init__.py
      __main__.py
      analytics.py
      benchmark.py
      data.py
      decorators.py
      processors.py
  storage/
    __init__.py
    __main__.py
    benchmark.py
    config.py
    context.py
    dataset.py
    dto.py
    exceptions.py
    exporters.py
    logging_config.py
    readers.py
    services.py
```

```

        validators.py
tests/
    test_analytics.py
    test_registry.py
    test_storage.py

```

Модуль	Призначення
exceptions.py	StorageError → ConfigurationError, DataError → Import/Export/UnsupportedFormat/RecordValidation
dto.py	StudentRow — форма файлу; конвертація у Student (ЛР1) і GradeRecord (ЛР2)
validators.py	перевірка id, name, group, grade, email; кожна помилка знає рядок і поле, причина збережена
config.py	config.yaml → frozen dataclasses, валідація до початку роботи, STUDENT_MANAGER_* override
logging_config.py	RotatingFileHandler + консоль, рівень із конфігурації
context.py	AtomicWriter (__enter__/__exit__), logged_operation (@contextmanager), open_text (.gz)
readers.py	Importer Protocol: CSV і JSON Lines потоково, JSON і YAML цілком; вибір за розширенням
exporters.py	Exporter Protocol: CSV, JSON, JSON Lines, YAML через AtomicWriter; StorageJSONEncoder
services.py	import_records зі strict/tolerant політикою, ImportStatistics, error_report, run_pipeline
dataset.py, benchmark.py	генератори демо- та великих файлів; експерименти strict/tolerant та eager/streaming
__main__.py	CLI: demo, run -c config.yaml (код виходу 0/1), experiments; console script student-storage

5. Ієрархія винятків

Усі винятки пакета успадковують StudentManagerError із ЛР1, тому код, який уже ловить базовий клас проекту, продовжує працювати, а новий код може бути настільки конкретним, наскільки треба. Дві гілки: проблеми конфігурації (fail-fast, до початку роботи) і проблеми з даними — не вдалося прочитати, не вдалося записати, невідомий формат, невалідний запис.

```

StudentManagerError                                (student_manager.exceptions, lab 1)
└─ StorageError
    └─ ConfigurationError                        config file missing / malformed / fails validation
        └─ DataError
            └─ DataImportError                    input cannot be opened or parsed
                └─ DataExportError                output cannot be written (old output stays intact)
                    └─ UnsupportedFormatError    extension maps to no known format
                        └─ RecordValidationError one record is wrong; knows the line and the field
"""

```

RecordValidationError пам'ятає номер рядка і поле: саме з них складаються повідомлення в журналі та рядки error CSV.

```

class RecordValidationError(DataError):
    """One record failed validation. Remembers where (line) and what (field) for the log and the error CSV."""

    def __init__(self, message: str, *, line_number: int | None = None, field: str | None = None) -> None:
        super().__init__(message)
        self.line_number = line_number
        self.field = field

    @property
    def location(self) -> str:
        """'рядок 12, поле grade' – for log messages and the error report."""
        where = "рядок ?" if self.line_number is None else f"рядок {self.line_number}"
        return f"{where}, поле {self.field}" if self.field else where

```

6. Валідація та exception chaining

Валідація працює за EAFP: замість перевіряти рядок регулярним виразом «чи це число», код викликає float() і перетворює ValueError на доменну помилку, зберігаючи причину через raise ... from error. Тоді traceback показує обидва рівні: що сталося насправді і що це означає для програми.

```

def parse_grade(value: object, line_number: int, field: str = "grade") -> float:
    """float() with a decimal comma allowed; NaN and infinities fail the range check like any other number."""
    try:
        grade = float(text_of(value).replace(",", "."))
    except ValueError as error:
        raise RecordValidationError(f"{field} має бути числом, отримано {text_of(value)!r}",
                                    line_number=line_number, field=field) from error
    if not GRADE_MIN <= grade <= GRADE_MAX:
        raise RecordValidationError(f"{field} має бути в межах {GRADE_MIN:g}..{GRADE_MAX:g}, отримано {grade:g}",
                                    line_number=line_number, field=field)
    return grade

def parse_student_row(row: Row, line_number: int) -> StudentRow:
    """Parse and validate one raw row; the domain model gets the last word (group pattern, grade range)."""
    student_id = parse_id(row, line_number)
    name = parse_required_text(row, "name", line_number)
    if len(name.split()) < 2:

```

```

        raise RecordValidationError(f"name має містити прізвище та ім'я, отримано {name!r}",
                                   line_number=line_number, field="name")
    group = parse_required_text(row, "group", line_number)
    grade = parse_grade(row.get("grade"), line_number)
    email = parse_required_text(row, "email", line_number)
    if not EMAIL_PATTERN.fullmatch(email):
        raise RecordValidationError(f"email {email!r} не схожий на адресу", line_number=line_number,
                                   field="email")
    dto = StudentRow(student_id, name, group, grade, email)
    to_student(dto, line_number=line_number)
    return dto

```

```
RecordValidationError: grade має бути числом, отримано 'wrong'
__cause__ = ValueError("could not convert string to float: 'wrong'")
Traceback:
Traceback (most recent call last):
  File "C:\Users\user\vscode\uni\python\project-lab5\src\student_manager\storage\validators.py", line
42, in parse_grade
    grade = float(text_of(value).replace(", ", "."))
ValueError: could not convert string to float: 'wrong'
```

[illegible]

7. Конфігурація

```
# Конфігурація демонстрації ЛР5. Шляхи – відносно цього файла.
version: 1
```

```

output:
  path: output/students.json
  errors_path: output/errors.csv
  summary_path: output/summary.json
  backup: true

processing:
  minimum_grade: 80
  skip_invalid: true          # tolerant mode; false = strict
  progress_every: 5

logging:
  level: INFO
  path: logs/storage.log
  max_bytes: 200000
  backup_count: 2

```

Завантаження — чотири кроки: прочитати YAML через `safe_load` (три різні низькорівневі помилки — три повідомлення, причина збережена), накласти `override` зі змінних середовища `STUDENT_MANAGER_*`, зібрати frozen dataclasses (відсутній ключ або не той тип — одна `ConfigurationError`), перевірити все, що можна перевірити до довгого імпорту: межі оцінки, наявність вхідного файлу, існування каталогу результату. Помилка в конфігурації коштує мілісекунду, а не аварію посеред роботи.

```

def read_yaml_mapping(path: Path) -> dict[str, Any]:
    """The raw document. Three different low-level failures, three messages, every cause kept."""
    try:
        with path.open("r", encoding="utf-8") as file:
            raw = yaml.safe_load(file)
    except FileNotFoundError as error:
        raise ConfigurationError(f"файл конфігурації не знайдено: {path}") from error
    except OSError as error:
        raise ConfigurationError(f"не вдалося прочитати конфігурацію {path}") from error
    except yaml.YAMLError as error:
        raise ConfigurationError(f"некоректний YAML у {path.name}: {str(error).splitlines()[0]}") from error
    if not isinstance(raw, dict):
        raise ConfigurationError(f"{path.name}: конфігурація має бути YAML-mapping, а не {type(raw).__name__}")
    return raw

def validate_config(config: AppConfig) -> None:
    """Everything that can be checked before the long import is checked here, so a typo fails in a millisecond."""
    if config.version != SCHEMA_VERSION:
        raise ConfigurationError(f"версія схеми конфігурації {config.version} не підтримується (очікую {SCHEMA_VERSION})")
    minimum = config.processing.minimum_grade
    if not GRADE_MIN <= minimum <= GRADE_MAX:
        raise ConfigurationError(f"minimum_grade має бути в межах {GRADE_MIN:g}..{GRADE_MAX:g}, отримано {minimum:g}")
    if config.processing.progress_every < 0:
        raise ConfigurationError("progress_every не може бути від'ємним")
    if config.logging.level not in LOG_LEVELS:
        raise ConfigurationError(f"невідомий рівень логування {config.logging.level!r}; допустимі: {'',

```

```

'.join(sorted(LOG_LEVELS))}")
    if config.logging.max_bytes <= 0 or config.logging.backup_count < 0:
        raise ConfigurationError("logging.max_bytes має бути додатним, backup_count – невід'ємним")
    if not config.input_path.is_file():
        raise ConfigurationError(f"вхідний файл не знайдено: {config.input_path}")
    if not config.output.path.parent.is_dir():
        raise ConfigurationError(f"каталог для результату не існує: {config.output.path.parent}")

def load_config(path: Path, *, environ: Mapping[str, str] | None = None) -> AppConfig:
    """Read -> override from the environment -> build -> validate. `environ` is injectable for tests
    and demos."""
    raw = read_yaml_mapping(path)
    apply_env_overrides(raw, os.environ if environ is None else environ)
    config = build_config(raw, path.parent)
    validate_config(config)
    return config

```

2. КОНФІГУРАЦІЯ: load_config -> AppConfig (YAML, frozen dataclasses, валідація)

```

-----
version=1 input=students.csv
output: path=output\students.json errors_path=output\errors.csv summary_path=output\summary.json
backup=True
processing: minimum_grade=80 skip_invalid=True progress_every=5
logging: level=INFO path=logs\storage.log max_bytes=200000 backup_count=2
з environment (MINIMUM_GRADE=90, SKIP_INVALID=no): minimum_grade=90, skip_invalid=False

```

9. ПОМИЛКИ КОНФІГУРАЦІЇ: fail-fast (експеримент 5)

```

-----
1. відсутній config.yaml      ConfigurationError: файл конфігурації не знайдено:
C:\Users\user\vscode\uni\python\project-lab5\data\broken_1.yaml <- FileNotFoundError
2. неправильний YAML         ConfigurationError: некоректний YAML у broken_2.yaml: while parsing a
flow sequence <- ParserError
3. немає ключа input         ConfigurationError: у конфігурації бракує ключа 'input' <- KeyError
4. minimum_grade = 150       ConfigurationError: minimum_grade має бути в межах 0..100, отримано
150
5. input file відсутній      ConfigurationError: вхідний файл не знайдено:
C:\Users\user\vscode\uni\python\project-lab5\data\missing.csv
6. output directory не існує  ConfigurationError: каталог для результату не існує:
C:\Users\user\vscode\uni\python\project-lab5\data\nowhere
2026-09-08 17:33:42 | ERROR    | student_manager.storage.main | критична помилка: файл конфігурації не
знайдено: C:\Users\user\vscode\uni\python\project-lab5\data\broken_1.yaml (причина: FileNotFoundError:
[Errno 2] No such file or directory: 'C:\\Users\\user\\vscode\\uni\\python\\project-
lab5\\data\\broken_1.yaml')
run(broken_1.yaml) -> exit code 1

```

8. Logging

Кореневий logger налаштовується з конфігурації: RotatingFileHandler з обмеженням розміру та кількістю резервних файлів плюс необов'язковий потік у консоль. Рівень — із YAML. print() для подій не використовується: у журналі є час, рівень, модуль і повідомлення, а logger.exception() додає traceback без зусиль.

```

def configure_logging(level: str, path: Path, *, max_bytes: int = 1_000_000, backup_count: int = 3,
                     console: TextIO | None = None) -> logging.Logger:
    """(Re)configure the root logger.

```

force=True closes and replaces the previous handlers, so a second call in the same process (tests,

the demo) does not duplicate every line. The log directory is created on demand: a missing logs/ is not a reason to refuse to run, unlike a missing input file.

```
"""
path.parent.mkdir(parents=True, exist_ok=True)
handlers: list[logging.Handler] = [
    RotatingFileHandler(path, maxBytes=max_bytes, backupCount=backup_count, encoding="utf-8"),
]
if console is not None:
    handlers.append(logging.StreamHandler(console))
logging.basicConfig(level=logging.getLevelNamesMapping()[level.upper()], format=LOG_FORMAT,
                    datefmt=DATE_FORMAT, handlers=handlers, force=True)
logger = logging.getLogger("student_manager.storage")
logger.debug("logging: рівень %s, файл %s", level.upper(), path)
return logger
```

12. ФРАГМЕНТ LOG-ФАЙЛУ

```
-----
logs\storage.log: 58 рядків, останні 10:
2026-09-08 17:33:42 | ERROR      | student_manager.storage.main | критична помилка: файл конфігурації
не знайдено: C:\Users\user\vscode\uni\python\project-lab5\data\broken_1.yaml (причина:
FileNotFoundError: [Errno 2] No such file or directory: 'C:\\Users\\user\\vscode\\uni\\python\\project-
lab5\\data\\broken_1.yaml')
2026-09-08 17:33:42 | INFO       | student_manager.storage.services | імпорт grades.csv, режим strict
2026-09-08 17:33:42 | INFO       | student_manager.storage.services | імпорт grades.jsonl, режим strict
2026-09-08 17:33:42 | INFO       | student_manager.storage.services | імпорт grades.yaml, режим strict
2026-09-08 17:33:42 | INFO       | student_manager.storage.services | імпорт grades.json.gz, режим
strict
2026-09-08 17:33:42 | INFO       | student_manager.storage.services | імпорт broken.json, режим strict
2026-09-08 17:33:42 | INFO       | student_manager.storage.services | імпорт broken.yaml, режим strict
2026-09-08 17:33:42 | INFO       | student_manager.storage.services | імпорт wrong_header.csv, режим
strict
2026-09-08 17:33:42 | INFO       | student_manager.storage.services | імпорт missing.csv, режим strict
2026-09-08 17:33:42 | INFO       | student_manager.storage.services | імпорт grades.xml, режим strict
```

9. Поточковий імпорт, strict і tolerant

CsvImporter.read() — генератор над csv.DictReader: заголовок перевіряється відразу (fail-fast, бракує колонок — DataImportError), далі рядки віддаються по одному з номером рядка файлу. JSON Lines читається так само потоково; JSON-масив і YAML — цілком, бо формат не дозволяє інакше.

```
class CsvImporter:
    """csv.DictReader row by row; the file is never fully in memory. The header is checked first (fail-
fast)."""

    def __init__(self, required: tuple[str, ...] = ()) -> None:
        self.required = required

    def read(self, path: Path) -> Iterator[NumberedRow]:
        try:
            with open_text(path, "r") as file:
                reader = csv.DictReader(file)
                if reader.fieldnames is None:
                    raise DataImportError(f"{path.name}: порожній файл, немає рядка заголовка")
                missing = [name for name in self.required if name not in reader.fieldnames]
                if missing:
```

```

        raise DataImportError(f"{path.name}: у заголовку бракує колонок {'',
'.join(missing)}")
    for row in reader:
        yield reader.line_num, row
except OSError as error:
    raise DataImportError(f"не вдалося прочитати {path}") from error
except csv.Error as error:
    raise DataImportError(f"{path.name}: пошкоджений CSV – {error}") from error

```

Політику вирішує один об'єкт: `strict` перериває імпорт першим невалідним записом, `tolerant` логує, рахує в `ImportStatistics` за полями, віддає рядок у `error CSV` і йде далі. Сам `import_records()` — generic-генератор: один сирий рядок на вході, один перевірений об'єкт на виході, файл ніколи не в пам'яті цілком.

```

class ImportPolicy:
    """What happens to an invalid record: strict aborts on the first one, tolerant logs, counts and
    skips."""

    skip_invalid: bool
    progress_every: int = 0

    @property
    def mode(self) -> str:
        return "tolerant" if self.skip_invalid else "strict"

    @classmethod
    def from_config(cls, processing: ProcessingConfig) -> ImportPolicy:
        return cls(skip_invalid=processing.skip_invalid, progress_every=processing.progress_every)

STRICT = ImportPolicy(skip_invalid=False)
TOLERANT = ImportPolicy(skip_invalid=True)

def import_records(path: Path, parse: Callable[[Row, int], T], *, required: tuple[str, ...],
                  policy: ImportPolicy, statistics: ImportStatistics,
                  on_invalid: InvalidCallback | None = None) -> Iterator[T]:
    """Generic streaming import: one raw row in, one validated object out, never the whole file.

    try/except/else: the parser may raise, the policy decides, `else` counts and yields only the good
    ones.

    In strict mode a bare `raise` re-raises the very same exception with its chain intact.
    """
    logger.info("імпорт %s, режим %s", path.name, policy.mode)
    for line_number, raw in read_rows(path, required=required):
        statistics.total += 1
        if policy.progress_every and statistics.total % policy.progress_every == 0:
            logger.info("прочитано %d записів", statistics.total)
        try:
            record = parse(raw, line_number)
        except RecordValidationError as error:
            statistics.note_invalid(error)
            logger.warning("некоректний запис (%s): %s", error.location, error)
            if on_invalid is not None:
                on_invalid(error, raw)
            if policy.skip_invalid:

```

```

        continue
    raise
else:
    statistics.valid += 1
    yield record

def run_pipeline(config: AppConfig) -> PipelineResult:
    """One configured run. `finally` logs the counters even when strict mode aborts half-way."""
    statistics = ImportStatistics()
    policy = ImportPolicy.from_config(config.processing)
    exported = 0
    start = perf_counter()
    errors: AbstractContextManager[InvalidCallback | None] = (
        nullcontext(None) if config.output.errors_path is None else
        error_report(config.output.errors_path)
    )
    try:
        with logged_operation("імпорт/експорт студентів"), errors as on_invalid:
            rows = import_students(config.input_path, policy=policy, statistics=statistics,
            on_invalid=on_invalid)
            selected = filter_by_grade(rows, config.processing.minimum_grade)
            exported = write_rows((row.to_dict() for row in selected), config.output.path,
                                backup=config.output.backup, fieldnames=STUDENT_FIELDS)
    finally:
        logger.info("підсумок: %s exported=%d (minimum_grade=%g, режим %s)",
                    statistics, exported, config.processing.minimum_grade, policy.mode)
    result = PipelineResult(statistics, exported, config.output.path, perf_counter() - start)
    if config.output.summary_path is not None:
        write_summary(config.output.summary_path, config, result)
    return result

```

10. Експорт, атомарний запис і власні context managers

AtomicWriter — context manager класом: `__enter__` відкриває `<name>.tmp`, `__exit__` при успіху одним `os.replace()` підставляє його на місце результату (за потреби лишивши `.bak`), при винятку — видаляє тимчасовий файл і не чіпає попередній результат. Тому аварія посеред експорту не залишає наполовину записаного JSON. `logged_operation` — generator-based context manager через `contextlib.contextmanager`: логує початок, успіх із часом або помилку з `traceback`.

```

class AtomicWriter:
    """Write to <name>.tmp; on success the temp file replaces the target in one os.replace() call, on
    failure it is deleted and the previous output is never touched. Optionally keeps the old file as
    .bak.

    with AtomicWriter(path, backup=True) as file:
        file.write(...)          # any exception here -> no half-written output
    """

    def __init__(self, path: Path, *, backup: bool = False) -> None:
        self.path = path
        self.backup = backup
        self.temporary = path.with_name(path.name + ".tmp")
        self._file: TextIO | None = None

    def __enter__(self) -> TextIO:

```

```

try:
    self._file = open_text(self.temporary, "w", compressed=self.path.suffix == ".gz")
except OSError as error:
    raise DataExportError(f"не вдалося створити тимчасовий файл {self.temporary}") from error
return self._file

def __exit__(self, exc_type: type[BaseException] | None, exc: BaseException | None,
             traceback: TracebackType | None) -> None:
    """Returns None (falsy), so the caller's exception is never swallowed."""
    if self._file is not None:
        self._file.close()
    if exc_type is not None:
        self.temporary.unlink(missing_ok=True)
        logger.warning("запис %s перервано (%s): тимчасовий файл видалено, попередній результат збережено",
                       self.path.name, exc_type.__name__)
    return

try:
    if self.backup and self.path.exists():
        self.path.replace(self.path.with_name(self.path.name + ".bak"))
        self.temporary.replace(self.path)
except OSError as error:
    self.temporary.unlink(missing_ok=True)
    raise DataExportError(f"не вдалося замінити {self.path} новим файлом") from error

```

@contextmanager

```

def logged_operation(name: str, *, log: logging.Logger = logger) -> Iterator[None]:
    """Log the start, then either the success with the elapsed time or the failure with its traceback.

```

The code before `yield` is `__enter__`, the code after it is `__exit__`; the exception raised inside the with-block shows up right at the yield, which is why the try/except wraps it.

```

"""
log.info("%s: початок", name)
start = perf_counter()
try:
    yield
except Exception:
    log.exception("%s: перервано через помилку після %.3f с", name, perf_counter() - start)
    raise
else:
    log.info("%s: завершено за %.3f с", name, perf_counter() - start)

```

JSON-експортер пише масив потоково — [, по одному об'єкту,] — не збираючи список; StorageJSONEncoder вчить json серіалізувати Path, dataclass, set і datetime.

```

class JsonExporter:
    """A JSON array written as '[', one object per record, ']' - the list itself is never built."""

    def __init__(self, *, backup: bool = False) -> None:
        self.backup = backup

    def write(self, rows: Iterable[Row], path: Path) -> int:
        count = 0
        try:

```

```

        with AtomicWriter(path, backup=self.backup) as file:
            file.write("[\n")
            for row in rows:
                if count:
                    file.write(",\n")
                file.write(indent(to_json(row, pretty=True), " "))
                count += 1
            file.write("\n]\n")
    except OSError as error:
        raise DataExportError(f"не вдалося записати {path}") from error
    except (TypeError, ValueError) as error:
        raise DataExportError(f"{path.name}: запис {count + 1} не серіалізується в JSON – {error}")
    from error
    return count

```

8. ATOMIC OUTPUT: збій посеред експорту (експеримент 4), потім успішний експорт із backup

```

-----
2026-09-08 17:33:42 | WARNING | student_manager.storage.context | запис students.json перервано
(OSError): тимчасовий файл видалено, попередній результат збережено
DataExportError: не вдалося записати C:\Users\user\vscode\uni\python\project-
lab5\data\output\students.json
__cause__ = OSError(28, 'No space left on device (імітація)')
students.json незмінний: True; .tmp існує: False; .bak існує: False
повторний експорт 6 записів із backup=True: .bak існує: True, збігається з попереднім: True

```

Пошкоджені вхідні файли перетворюються на `DataImportError` зі збереженою причиною, невідоме розширення — на `UnsupportedFormatError`; той самий набір `GradeRecord` із `LP2` проходить туди й назад через усі чотири формати без втрат.

10. ФОРМАТИ: `GradeRecord` (LP2) -> CSV / JSON Lines / YAML / JSON.gz -> назад (Importer/Exporter Protocol)

```

-----
--
2026-09-08 17:33:42 | INFO | student_manager.storage.services | імпорт grades.csv, режим strict
grades.csv записано 36, прочитано 36, 2496 байт, збігається з оригіналом: True
2026-09-08 17:33:42 | INFO | student_manager.storage.services | імпорт grades.jsonl, режим strict
grades.jsonl записано 36, прочитано 36, 4372 байт, збігається з оригіналом: True
2026-09-08 17:33:42 | INFO | student_manager.storage.services | імпорт grades.yaml, режим strict
grades.yaml записано 36, прочитано 36, 3976 байт, збігається з оригіналом: True
2026-09-08 17:33:42 | INFO | student_manager.storage.services | імпорт grades.json.gz, режим strict
grades.json.gz записано 36, прочитано 36, 563 байт, збігається з оригіналом: True
grades.jsonl:
{"student": "Гнатишин Марта", "group": "ФЕП-31с", "discipline": "Професійний Python", "grade": 95.0}
{"student": "Гнатишин Марта", "group": "ФЕП-31с", "discipline": "Мова С", "grade": 91.0}
... (ще 34 рядків)
grades.yaml:
- student: Гнатишин Марта
  group: ФЕП-31с
  discipline: Професійний Python
  grade: 95.0
... (ще 140 рядків)

```

11. ПОШКОДЖЕНІ ФАЙЛИ: `DataImportError` із збереженою причиною

```

-----
2026-09-08 17:33:42 | INFO | student_manager.storage.services | імпорт broken.json, режим strict
broken.json DataImportError: broken.json: некоректний JSON – Expecting value (рядок 1)
<- JSONDecodeError: Expecting value: line 1 column 102 (char 101)
2026-09-08 17:33:42 | INFO | student_manager.storage.services | імпорт broken.yaml, режим strict

```

```

broken.yaml      DataImportError: broken.yaml: некоректний YAML – while parsing a flow sequence
                  <- ParserError: while parsing a flow sequence
                  in "C:\Users\user\vscode\uni\python\project-lab5\data\broken.yaml", line 2, column 10
                  expected ',' or ']', but got '<stream end>'
                  in "C:\Users\user\vscode\uni\python\project-lab5\data\broken.yaml", line 3, column 1
2026-09-08 17:33:42 | INFO      | student_manager.storage.services | импорт wrong_header.csv, режим
strict
                  wrong_header.csv DataImportError: wrong_header.csv: у заголовку бракує колонок student, group,
discipline, grade
2026-09-08 17:33:42 | INFO      | student_manager.storage.services | импорт missing.csv, режим strict
missing.csv      DataImportError: не вдалося прочитати C:\Users\user\vscode\uni\python\project-
lab5\data\missing.csv
                  <- FileNotFoundError: [Errno 2] No such file or directory:
'C:\\Users\\user\\vscode\\uni\\python\\project-lab5\\data\\missing.csv'
2026-09-08 17:33:42 | INFO      | student_manager.storage.services | импорт grades.xml, режим strict
grades.xml       UnsupportedFormatError: невідомий формат файлу 'grades.xml': очікую .csv, .json,
.jsonl, .yaml (можна з .gz)
                  <- KeyError: '.xml'

```

11. Результати запуску

11.1. Tolerant mode: python -m student_manager.storage run -c data/config.yaml

```

2026-09-08 17:33:42 | INFO      | student_manager.storage.context | импорт/экспорт студентів: початок
2026-09-08 17:33:42 | INFO      | student_manager.storage.services | импорт students.csv, режим tolerant
2026-09-08 17:33:42 | INFO      | student_manager.storage.services | прочитано 5 записів
2026-09-08 17:33:42 | INFO      | student_manager.storage.services | прочитано 10 записів
2026-09-08 17:33:42 | WARNING   | student_manager.storage.services | некоректний запис (рядок 11, поле
name): поле name порожнє
2026-09-08 17:33:42 | WARNING   | student_manager.storage.services | некоректний запис (рядок 12, поле
grade): grade має бути в межах 0..100, отримано 105
2026-09-08 17:33:42 | WARNING   | student_manager.storage.services | некоректний запис (рядок 13, поле
student_id): student_id має бути цілим числом, отримано 'abc'
2026-09-08 17:33:42 | WARNING   | student_manager.storage.services | некоректний запис (рядок 14, поле
grade): grade має бути числом, отримано 'wrong'
2026-09-08 17:33:42 | WARNING   | student_manager.storage.services | некоректний запис (рядок 15, поле
email): email 'olena.shevchuk-at-lnu' не схожий на адресу
2026-09-08 17:33:42 | INFO      | student_manager.storage.services | прочитано 15 записів
2026-09-08 17:33:42 | WARNING   | student_manager.storage.services | некоректний запис (рядок 16, поле
group): назва групи '31c' не схожа на ФЕП-31c
2026-09-08 17:33:42 | INFO      | student_manager.storage.context | импорт/экспорт студентів: завершено
за 0.015 с
2026-09-08 17:33:42 | INFO      | student_manager.storage.services | підсумок: total=15 valid=9
invalid=6 exported=6 (minimum_grade=80, режим tolerant)
Готово: total=15 valid=9 invalid=6, exported=6 -> data\output\students.json
exit code: 0

```

Дев'ять валідних записів із п'ятнадцяти, шість відхилених — по одному на кожен вид дефекту, — і після фільтра `minimum_grade: 80` у результат потрапило шість. Поряд із `students.json` лежать `errors.csv` (рядок, поле, повідомлення, сирий запис) і `summary.json` (конфігурація, статистика, час).

```

students.json (6 записів):
[
  {
    "student_id": 1,

```

```

    "name": "Гнатишин Марта",
    "group": "ФЕП-31с",
    "grade": 93.25,
    "email": "marta.hnatyshyn@lnu.edu.ua"
  },
  {
    "student_id": 3,
    "name": "Бовк Тарас",
    "group": "ФЕП-31с",
    "grade": 85.0,
    "email": "taras.vovk@lnu.edu.ua"
  }
  ... (ще 30 рядків)
errors.csv:
line,field,message,raw
11,name,поле name порожнє,"{"student_id": "10", "name": "", "group": "ФЕП-31с",
"grade": "76.0", "email": "noname@lnu.edu.ua"}"
12,grade,"grade має бути в межах 0..100, отримано 105","{"student_id": "11", "name": "Новак
Петро", "group": "ФЕП-33", "grade": "105", "email": "petro.novak@lnu.edu.ua"}"
... (ще 4 рядків)
summary.json: statistics={'total': 15, 'valid': 9, 'invalid': 6, 'invalid_by_field': {'name': 1,
'grade': 2, 'student_id': 1, 'email': 1, 'group': 1}}, exported=6

```

11.2. Strict mode: перша помилка зупиняє імпорт

```

2026-09-08 17:33:43 | INFO      | student_manager.storage.context | імпорт/експорт студентів: початок
2026-09-08 17:33:43 | INFO      | student_manager.storage.services | імпорт students.csv, режим strict
2026-09-08 17:33:43 | INFO      | student_manager.storage.services | прочитано 5 записів
2026-09-08 17:33:43 | INFO      | student_manager.storage.services | прочитано 10 записів
2026-09-08 17:33:43 | WARNING   | student_manager.storage.services | некоректний запис (рядок 11, поле
name): поле name порожнє
2026-09-08 17:33:43 | WARNING   | student_manager.storage.context | запис students.json перервано
(RecordValidationError): тимчасовий файл видалено, попередній результат збережено
2026-09-08 17:33:43 | WARNING   | student_manager.storage.context | запис errors.csv перервано
(RecordValidationError): тимчасовий файл видалено, попередній результат збережено
2026-09-08 17:33:43 | ERROR      | student_manager.storage.context | імпорт/експорт студентів: перервано
через помилку після 0.013 с

2026-09-08 17:33:43 | INFO      | student_manager.storage.services | підсумок: total=10 valid=9
invalid=1 exported=0 (minimum_grade=80, режим strict)
2026-09-08 17:33:43 | ERROR      | student_manager.storage.main | критична помилка: поле name порожнє
exit code: 1

```

Код виходу 1, у журналі — повний traceback із ланцюжком, у консолі користувача — одне повідомлення. Тимчасові файли видалено, попередній `students.json` не змінився (демонстрація перевіряє це явно).

11.3. Критична помилка конфігурації

```

критична помилка: файл конфігурації не знайдено: data\missing.yaml (причина: FileNotFoundError: [Errno
2] No such file or directory: 'data\missing.yaml')
exit code: 1

```

11.4. Доменна модель

Результат експорту читається назад і потрапляє в `StudentRegistry` із ЛР1 — форми файлу і домену з'єднує DTO `StudentRow`, тож жодній зі сторін не довелося підлаштовуватися.

5. ДОМЕННА МОДЕЛЬ: `students.json` -> `StudentRow` -> `Student` -> `StudentRegistry` (ЛР1)

zareestrovano 6; naykrashiy: Pasichnik Irina (ФЕП-32, 96.50)
ФЕП-31с: 2 студ., середній бал 89.12
ФЕП-32: 2 студ., середній бал 92.25
ФЕП-33: 2 студ., середній бал 86.50

12. Експериментальна частина

12.1. Strict проти tolerant

Один файл: 1 000 валідних записів і 10 невалідних, рівномірно розкиданих. Той самий код, різна лише політика.

Параметр	Strict	Tolerant
Оброблено записів	101	1010
Valid	100	1000
Invalid	1	10
Exported	0	1000
Завершення	перервано (рядок 102, поле grade)	завершено успішно
Файл результату	немає	є

Strict зупинився на 102-му записі й не створив файла результату; tolerant дійшов до кінця, експортував 1 000 записів і зафіксував 10 відхилених у error CSV. Strict доречний, коли неповний результат гірший за відсутній (фінансові дані, міграції); tolerant — коли краще отримати 99 % даних зараз і список проблем поруч.

12.2. Eager проти streaming

list(csv.DictReader(...)) проти генератора над тим самим reader; час без tracemalloc окремо від пікової пам'яті з ним.

Записів	Eager, c	Streaming, c	Eager, MiB	Streaming, MiB	Відношення
10 000	0.145	0.117	4.85	0.07	68×
100 000	1.432	1.196	48.28	0.07	676×
500 000	7.697	6.160	241.86	0.07	3 389×

Пам'ять eager-варіанта росте лінійно — близько 0.5 KB на запис, 242 MiB на півмільйона; потоковий варіант тримає сталі 70 KiB незалежно від розміру файла. Час потокового читання ще й на 15–20 % менший: не треба виділяти й розширювати величезний список. Це й є відповідь, чому list(reader) — погана звичка: вона не дає нічого, крім лінійної пам'яті.

13. Тести та перевірка типів

Пакет покрито 36 тестами у `tests/test_storage.py`: валідація кожного поля з перевіркою `__cause__`, fail-fast заголовка CSV, chaining для відсутнього файлу, зламаних JSON і YAML, round-trip усіх форматів, gzip, custom encoder, атомарний запис при успіху, при збої та при відсутньому каталозі, error report, override зі змінних середовища, кожна помилка конфігурації, ротація журналу, strict/tolerant, лінівисть імпорту, конвеєр наскрізь і збереження старого результату. Разом із попередніми роботами — 76 тестів; туру без зауважень.

```
..... [ 94%]
.... [100%]
76 passed in 0.82s

Success: no issues found in 27 source files
```

14. Робота з Git

Шість комітів у гілці lab5: винятки, DTO і валідатори; конфігурація, logging і context managers; імпортери й експортери; конвеєр зі статистикою й демо-даними; CLI та експерименти; тести й README.

```
7c8a5be Add storage tests, README section and console script, bump version to 0.5.0
e82a917 Add storage CLI with run/demo/experiments, strict vs tolerant and eager vs streaming benchmarks
246f3fe Add import/export pipeline with strict/tolerant policy, statistics, error report and demo data set
03d1cfc Add streaming importers and atomic exporters for CSV, JSON, JSON Lines and YAML
6cd5b12 Add YAML configuration loader with validation, logging setup and context managers
686bc45 Add storage package: exception hierarchy, DTO layer and record validators
ec3e1ec Fix group name in README
9770176 Wire analytics demo into CLI, add tests, bump version to 0.2.0
94e46bd Add list vs dict search benchmark

On branch lab5
nothing to commit, working tree clean
```

15. Відповіді на контрольні запитання

1. Що таке exception?

Об'єкт-сигнал про виняткову ситуацію, який перериває звичайний потік виконання, поки його хтось не перехопить.

2. Для чого використовується try?

Позначити блок, у якому помилка очікувана і буде оброблена.

3. Для чого ехсерт?

Перехопити виняток зазначеного типу і виконати обробку.

4. Коли виконується else після try?

Лише якщо блок `try` завершився без винятку.

5. Коли виконується `finally`?

Завжди — і після успіху, і після винятку, і після `return`.

6. Для чого використовується `raise`?

Кинути виняток: сигналізувати про помилку вгору по стеку.

7. Що означає `raise` без аргументів?

Повторно кинути виняток, який зараз обробляється, зберігши його `traceback`.

8. Чому не рекомендується `except: pass`?

Ховає будь-яку помилку, включно з `KeyboardInterrupt` і помилками програміста; код «працює», а дані тихо втрачаються.

9. Чому потрібно перехоплювати конкретні `exception types`?

Щоб обробляти лише те, що ви вмієте обробити, а решта (баги) падала гучно.

10. Що таке `exception hierarchy`?

Дерево класів винятків: базовий клас проєкту, під ним категорії, під ними конкретні помилки.

11. Для чого створювати `custom exceptions`?

Щоб код клієнта ловив доменні помилки за змістом (`ConfigurationError`), а не за низькорівневим типом (`KeyError`).

12. Що таке `exception chaining`?

Зв'язування нового винятку з тим, що його спричинив: `__cause__` або `__context__`.

13. Що означає `raise ... from ...`?

Явно задати причину: новий виняток із `__cause__` = старий; `traceback` покаже обидва.

14. Яку перевагу дає `chaining` для `debugging`?

Видно і що сталося насправді (`ValueError` від `float`), і що це означає (`grade` має бути числом, рядок 14).

15. Що означає EAFP?

Easier to Ask Forgiveness than Permission: спробувати і обробити виняток замість попередніх перевірок.

16. Що означає LBYL?

Look Before You Leap: перевірити умови перед дією.

17. Що таке fail-fast?

Виявляти помилку якнайраніше і зупинятися, поки вона не наробила шкоди.

18. Де доцільно застосовувати fail-fast?

Конфігурація, заголовок файла, аргументи запуску — все, що можна перевірити до довгої роботи.

19. Що таке context manager?

Об'єкт із `__enter__`/`__exit__`, який гарантує звільнення ресурсу після блоку `with`.

20. Для чого використовується with?

Виконати блок із гарантованим прибиранням — закриттям файла, транзакцією, замком.

21. Які methods утворюють context manager protocol?

`__enter__()` і `__exit__(exc_type, exc, tb)`.

22. Що робить __enter__()?

Готує ресурс і повертає те, що піде в `as`.

23. Що робить __exit__()?

Прибирає ресурс; отримує інформацію про виняток і може його погасити, повернувши `True`.

24. Чи закриває with файл при exception?

Так, `__exit__` викликається і при винятку — файл закривається завжди.

25. Для чого contextlib.contextmanager?

Написати context manager як генератор з одним `yield` замість класу.

26. Яку роль має yield у generator-based context manager?

Розділяє код `__enter__` (до `yield`) і `__exit__` (після); значення `yield` іде в `as`.

27. Що таке pathlib.Path?

Об'єктний шлях з операціями `/`, `.exists()`, `.open()`, `.with_suffix()` тощо.

28. Чому pathlib кращий за ручну конкатенацію шляхів?

Правильні роздільники на всіх ОС, без помилок зі слешами, методи замість `os.path`-функцій.

29. Для чого задається encoding="utf-8"?

Щоб файл читався однаково на будь-якій системі; типове кодування Windows — cp1251, і кирилиця ламається.

30. Що таке CSV?

Текстовий табличний формат: рядки записів, поля через кому, перший рядок — заголовок.

31. Для чого csv.reader?

Розбирати CSV на списки полів з урахуванням лапок і екранування.

32. Для чого csv.DictReader?

Те саме, але кожен рядок — dict за назвами колонок із заголовка.

33. Чим DictReader зручний для structured data?

Доступ за іменем поля, а не індексом; порядок колонок у файлі не важливий.

34. Для чого csv.DictWriter?

Писати dict-записи у CSV із заданим заголовком.

35. Що таке JSON?

Текстовий формат обміну даними: об'єкти, масиви, рядки, числа, true/false/null.

36. Що таке serialization?

Перетворення об'єктів у пам'яті на послідовність байтів/текст для збереження чи передачі.

37. Що таке deserialization?

Зворотне відновлення об'єктів із збереженого подання.

38. Чим json.dump() відрізняється від json.dumps()?

dump пише у файловий об'єкт, dumps повертає рядок.

39. Чим json.load() відрізняється від json.loads()?

load читає з файлового об'єкта, loads — із рядка.

40. Що таке JSONDecodeError?

Виняток при розборі некоректного JSON; містить позицію помилки.

41. Для чого використовується YAML?

Зручні для людини конфігурації: відступи, коментарі, вкладені структури.

42. Чому рекомендується yaml.safe_load()?

`yaml.load` може створювати довільні Python-об'єкти з ненадійного файла; `safe_load` — лише базові типи.

43. Які типові сценарії використання JSON?

API, обмін між сервісами, збереження структурованих даних, логи в JSON Lines.

44. Які типові сценарії використання YAML?

Конфігурації застосунків, CI/CD, Docker Compose, Kubernetes.

45. Що таке configuration file?

Зовнішній файл із параметрами програми, які змінюються без правки коду.

46. Чому configuration краще не hard-code у Python?

Різні середовища мають різні шляхи й пороги; зміна не має вимагати нового релізу.

47. Що потрібно перевіряти у configuration?

Обов'язкові ключі, типи, межі значень, існування вхідних файлів і каталогів, версію схеми.

48. Що таке logging?

Стандартний механізм запису подій програми з рівнями, форматом і призначенням.

49. Чим logging відрізняється від print()?

Рівні, час і модуль автоматично, кілька призначень, увімкнення/вимкнення без правки коду.

50. Які існують logging levels?

DEBUG, INFO, WARNING, ERROR, CRITICAL.

51. Для чого logger.info()?

Звичайні події: початок, прогрес, підсумок.

52. Для чого logger.warning()?

Щось не так, але робота триває — відхилений запис.

53. Для чого logger.error()?

Операція не вдалася.

54. Для чого logger.exception()?

Те саме, що error, але з traceback поточного винятку; викликати всередині ехсепт.

55. Що містить traceback?

Ланцюжок викликів від точки входу до місця помилки з файлами, рядками та причинами.

56. Чому secrets не можна записувати в logs?

Журнали читають багато людей і систем, зберігаються довго і потрапляють у бекапи.

57. Що take streaming I/O?

Обробка файла порціями в міру читання, без завантаження цілком.

58. Яка перевага streaming для великих files?

Стала пам'ять — 70 KiB замість 242 MiB на 500 000 записів у експерименті.

59. Що take eager file loading?

Прочитати весь файл (read() або list(reader)) перед обробкою.

60. Чому list(reader) може бути небажаним?

Матеріалізує всі записи; пам'ять росте лінійно, а виграшу в часі немає.

61. Що take strict validation mode?

Перший невалідний запис перериває імпорт; результат або повний, або відсутній.

62. Що take tolerant mode?

Невалідні записи логуються, рахуються й пропускаються; результат — усе валідне.

63. Коли доцільно використовувати strict mode?

Коли неповні дані гірші за відсутні: платежі, міграції, звітність.

64. Коли tolerant mode?

Коли джерело завідомо брудне і краще отримати 99 % зараз плюс список проблем.

65. Що take atomic write?

Запис у тимчасовий файл і заміна цільового одним os.replace(): результат або старий, або новий, ніколи не половинчастий.

66. Для чого використовується temporary file?

Щоб помилка посеред запису не зіпсувала попередній результат.

67. Що може статися при звичайному output write та аварійному завершенні?

Файл обрізаний на півдорозі — некоректний JSON, який наступний запуск не прочитає.

68. Навіщо видаляти temporary file після error?

Щоб не лишати сміття, яке хтось прийме за результат або яке з'їсть диск.

69. Що таке JSON Lines?

По одному JSON-об'єкту на рядок без зовнішнього масиву.

70. Чому JSON Lines зручний для streaming?

Рядок можна прочитати, розібрати й записати незалежно від решти файла — як CSV, але з типами.

16. Висновок

Створено модуль імпорту, експорту та конфігурації студентів за варіантом 1: ієрархія з шести винятків над базовим класом проєкту, валідація кожного поля з exception chaining, конфігурація YAML → frozen dataclasses з перевіркою до початку роботи та override зі змінних середовища, logging із ротацією, потокові імпортери CSV і JSON Lines, експортери чотирьох форматів через атомарний запис із резервною копією, два власні context managers, strict/tolerant політика зі статистикою та error CSV, CLI з кодами виходу. Експерименти показали, що strict дає або повний результат, або жодного, tolerant — результат плюс список проблем, а потокове читання тримає 70 KiB там, де eager бере 242 MiB. Модуль покрито 36 тестами, проходить туру, зафіксований шістьма комітами; наступна робота отримує від нього перевірені DTO для збереження в базі даних.